

Grundlagen

Postfix: x++, auslesen dann inkrementieren
Prefix: ++x, inkrementieren dann auslesen
Call by Value: Kopie von Wert/Referenz (Java kann nur das)
Call by Reference: Werte können verändert werden

Sichtbarkeit:

- **Public:** Alle Klassen
- **Protected:** Packages und alle Sub-Klassen
- **Private:** Nur eigene Klasse
- **Kein Keyword:** Alle Klassen im selben Package

Ternary Operator: x = a ? b : c → if(a){x=b;}else{x=c;}
Enhanced For-Statement: for(String s : stringList) { System.out.println(s); }

Primitive Datentypen

	boolean	Boolescher Wert	True, false
char	Textzeichen (UTF-16)	'a', 'B', '0', '!', etc.	
byte	Ganzzahl (8 Bit)	-128 bis 127	
short	Ganzzahl (16 Bit)	-32 768 bis 32 767	
int	Ganzzahl (32 Bit)	-2 ³¹ bis 2 ³¹ -1	
long	Ganzzahl (64 Bit)	-2 ⁶³ bis 2 ⁶³ -1, 1L (L Suffix)	
float	Gleitkommazahl (32 Bit)	0.1f, 2e4f (2 * 10 ⁴)	
double	Gleitkommazahl (64 Bit)	0.1, 2e4	

Arrays

Array erstellen: int[] a = new int[5]; a[2] = 1; Array ist Referenztyp
Arrays vergleichen (ganzer): Arrays.equals(a,b), Arrays.deepEquals(a,b)
Geschachtelte Arrays: Jede Zeile kann unterschiedlich lang sein, die zweite Dimension wird separat erzeugt: int[][] m = new int[3][5]; m[0] = new int[1];
Mehrdimensionales Array: Alle Zeilen haben die gleiche Länge: int[][] m = new int[2][3];

Casting

Explizit: Zu schmaleren Datentypen → Informationsverlust ist möglich
Implizit: Zu breiteren Datentyp (automatisch) → kein Informationsverlust

Methoden

public void print() { System.out.println("Hi"); } // bei void kein Rückgabety

Klassen/Objekte/Referenzen

Wertetypen: Variable enthält Daten direkt → int x = -5;
Referenztyp: Variable enthält Referenz auf Objekt → Point a = new Point();
public class Rectangle {
//Instanzvariablen
private final int length;
private int width;
//Klassenvariablen
private static int maxLength = 0;
//Konstruktor
public Rectangle(int length, int width) {
this.length = length; this.width = width; }
//Instanzmethoden
public int getWidth() { return width; }
//Klassenmethoden
public static int getMaxLength() { return maxLength; }
//Objekt Instanzieren
Rectangle r = new Rectangle(10, 20);

Satic/Listen/Wrapper-Klassen

Wrapper-Klassen: Sind keine primitiven Datentypen, Wert wird in Objekt gespeichert (int → Integer)
var-Deklaration: Verwenden bei Referenztypen, nicht bei primitiven Datentypen → var vehicle = new Motorcycle(); **var vehicle = null;** //ungültig
Static: Gehört zur Klasse, nicht zu einzelnen Objekten. Können nicht direkt auf Instanzvariablen zugreifen. z.B ClassName.print();

Vererbung

Abgeleitete Klasse erbt alle Members der Basisklasse (Instanzvariablen und Methoden).-> Lizkov *With super(speed)* kann Variable von Superklasse aufrufen werden. Muss als erstes stehen.
`class Car extends Vehicle { int doors = 5; void lock() {..}; @Override void drive([....])`
Override Annotation: @Override → um zu prüfen ob Methode in Basisklasse existiert

Polymorphismus

```
class Graphic {  
    void moveTo(Graphic other) {  
        System.out.println("Method 1");  
    }  
}  
  
class Circle extends Graphic {  
    void moveTo(Graphic other) {  
        System.out.println("Method 2");  
    }  
    void moveTo(Circle other) {  
        System.out.println("Method 3");  
    }  
}
```

Overloading: Auswahl nach Compile-Time Typen (rechts)

Overriding: Ausführung nach Runtime-Typ des Receivers (links)

1. Class des RT-Typ anschauen von moveTo(g)

2. Schauen, ob der RT-Typ von c.moveTo(g) in dieser Class ist, falls ja: CT anschauen und in diese Klasse gehen, Methode von RT g ausführen

falls nein: einen „Typ“ höher suchen, z.B. für Circle wird Graphic genommen und dann muss die spezifischere Methode ausgeführt werden (also in Circle Class).

Falls es in Class Graphic ein Objekt gäbe, wäre diese spezifischer.

Abstract

Implementierung kann bei Methoden fehlen. Klasse kann nicht instanziiert werden. **Keyword:** class Neu extends AbstractClass

```
abstract class Vehicle {  
    private int speed; public void accelerate() { ... }  
    public abstract void drive(); } → hat kein Rumpf, muss implementiert werden in Subklasse
```

Overloading and Overriding

Overloading: Methode mit gleichen Namen aber andere Parameter (unterschiedliche Anzahl oder Typen).
Regeln: Spezifischere Methode wählen

Konstruktor-Overloading: public class Rectangle { private int length; private int width; public Rectangle() { this(10, 20); } public Rectangle(int length, int width) { this.length = length; this.width = width; } }

Overriding: Methode mit identischer Signatur (gleich Name gleich Parameter) in Subklasse erneut implementiert. Mit *super.methode()* kann überschriebene Methode von Superklasse in Subklasse auferufen werden. Auflösung dynamisch (dynamic dispatch durch Laufzeitsystem)

Hiding: Subklasse definiert Instanzvariable mit gleichem Namen wie Superklasse neu.

Final Methode: Können nicht überschrieben werden.
Finale Klasse: Kann nicht durch Subklasse erweitert werden

Equals

==: Vergleiche nur Referenzen (ist es das gleiche Objekt)
a.equals(b): Vergleicht die Inhalte der Referenzen (wichtig bei String)

```
class Person { ...  
@Override  
public boolean equals(Object obj) {  
    if (obj == null) { return false; }  
    if (getClass() != obj.getClass()) { return false; }  
    Person other = (Person)obj;  
    return Objects.equals(firstName, other.firstName) &&  
    Objects.equals(lastName, other.lastName); }  
@Override public int hashCode() {  
    return Objects.hash(firstName, lastName); } }
```

Packages/Module/Enum

Innere Klassen: Wenn weitere Klassen in der Datei sind, sind diese nicht mehr public. Nicht-statische innere Klassen benötigen eine umgebende Instanz (... = top.new InnerMemberClass());

Packages

Bibliothek von Klassen, Interfaces etc. Definiert einen eigenen Namensraum. Erstellen mit *package p1*; Import mit *import p1.A*;
Sub-Packages können innerhalb eines Packages erstellt werden (werden nicht automatisch Importiert).
Import-Konflikte - Vorrang Reihenfolge:

1. Eigene Klasse (bzw. innere Klassen)
2. Single Type Imports (import p2.a;)
3. Typen in eigenem Package (package p1;)
4. Import on Demand (import p2.*;)

Enum

Eigener Datentyp mit endlichem Wertebereich (Auflistung)

```
public enum Weekday { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY; //müssen Grossbuchstaben sein  
public boolean isWeekend() { return this == SATURDAY || this == SUNDAY; }  
//Verwendung: Weekday currentDay = Weekday.WEDNESDAY; if (currentDay == Weekday.SUNDAY) { ... }
```

Enum mit Konstruktor:

```
public enum Weekday {  
    MONDAY(true), TUESDAY(true), WEDNESDAY(true), THURSDAY(true),  
    FRIDAY(true), SATURDAY(false), SUNDAY(false);  
    private boolean workDay;  
    Weekday(boolean workDay) { this.workDay = workDay; }  
    public boolean isWorkDay() { return workDay; } }
```

Interfaces

Merfchvererbung geht nicht, mit Interfaces jedoch schon.
Implementierungsdetails sind nicht Teil des Interfaces. Methoden sind **immer implizit public und abstract**. Interface ist implizit abstract.

interface Vehicle {
 int HIGHWAY_MIN_SPEED = 60; //Konstante müssen direkt Initialisiert werden und sind automatisch public static final
 void drive(); int maxSpeed(); }
//Implementation
class MobileHome implements Vehicle, House { ... }
//Schnittstelle kann andere Schnittstelle erweitern
interface RoadVehicle extends Vehicle, I1 { String tireModel(); }
//Mögliche Implementation
class C2 extends C1 implements I1, I2

Kollisionen:

1. Methoden mit gleichen Namen → @override nur einmal verwenden
2. Gleichnamige konstanten → RoadVehicle.MAX_SPEED & WaterVehicle.MAX_SPEED
3. Diverse Rückgabetyen:
interface RoadVehicle { String getModel(); }
interface WaterVehicle { int getModel(); }

Nicht Implementierbar ausser Rückgabetyen haben gemeinsamen Subtyp. Kann gelöst werden: @Override public AmbhbianMobile clone() { ... }
Default Methods: default void printModel([..]) → können überschrieben werden, es wird immer die spezifischere Default Implementation gewählt. Falls beide gleich muss manuell Konflikt behoben werden.

Collections

Iterator Interface (Iterable)

Jede Collection ist Iterable (Iterator oder Enhanced for loop verwendbar).
Iterable: Man kann iterieren, **Iterator:** Wie man iteriert (hasNext(), next())
Entfernen:
var stringList = new ArrayList<String>();
Iterator<String> it = stringList.iterator();
while (it.hasNext()) {
 String elem = it.next();
 if (elem.equals("..")) { it.remove(); } }

Lists

Geordnete Folge, Duplikate erlaubt. **Operationen:** add(), addAll(), clear(), contains(), get(), remove(), isEmpty()

	ArrayList	LinkedList
Index-Zugriff	Sehr schnell	Langsam
Einfügen	Langsam	Sehr schnell
Suchen	Langsam	Langsam
Speicher	Array mit Reserveplatz	Overhead pro Element

ArrayList: List firstList = new ArrayList<>();
LinkedList: List otherList = new LinkedList<>();
Stack (Last in, First Out), Operationen: push(), pop(), top(), size(), isEmpty(), isFull()
Queue (FIFO): Eine LinkedList, einfügen hinten, entfernen vorne
Deque: einfügen/entfernen geht vorne und hinten

Sets

Keine Duplikate, Menge von Elementen
HashSet: Sehr schnell, benötigt guten hashCode() (muss konsistent zu equals() sein)
Set firstSet = new TreeSet<>();
TreeSet: Sortiert, etwas langsamer, Elemente müssen Comparable und equals () implementieren
Set carModels = new TreeSet<>();

Maps

Schlüssel-Wert Paare, schnelles finden über Schlüssel, gleiche Regel wie bei Sets
HashMap: unsortiert, sehr schnell
Map<Integer, Student> map = new HashMap<>;
TreeMap: sortiert nach Schlüssel → Map masters = new HashMap<>;

Hashing

hashCode() muss schnell laufen und gleiches Resultat wie equals() erzeugen. Gute Hash-Funktion = wenige Kollisionen = hohe Performance

Exceptions

Exceptions behandeln: finally ist optional, wird in jedem Fall durchlaufen
try (String s = ...; String c = clip(s)) {
 catch (StringClipException e) { System.out.println("cannot clip"); }
 finally { System.out.println("finished"); }
} **Benutzerdefinierte Exceptions:**
class StringClipException extends Exception {
 public StringClipException() { }
 public StringClipException(String message) { super(message); }
} **Checked Exceptions:** Muss behandelt werden, muss deklariert werden, vom compiler geprüft (z.B Exception)
String clip(String s) throws NoStringException, ShortStringException {
 if (s == null) { throw new NoStringException(); }
 if (s.length() < 2) {
 throw new ShortStringException(); }
 return s.substring(1, s.length()-1); }
Unchecked Exceptions: Keine declaration und Behandlung nötig, nicht vom Compiler geprüft (z.B RuntimeException, Error)
String clip(String s) {
 if (s == null) {
 throw new NullPointerException("no string"); }
 if (s.length() < 2) {
 throw new IllegalArgumentException("short string"); }
 return s.substring(1, s.length()-1); }

Scanner: try (Scanner s = new Scanner(System.in)) { ... // work with s }

Input/Output

Byte Stream

Klassen: InputStream, OutputStream → gut für Bilder, PDFs etc
Lesen: FileInputStream, read() liest ein Byte, -1 bedeutet Ende der Datei
try (var in = new FileInputStream("data.bin")) {
 int b;
 while ((b = in.read()) != -1) {}
Schreiben: FileOutputStream, write(byte), neu erstellen oder überschreiben mit true
try (var out = new FileOutputStream("data.bin", true)) {out.write(42);}

Character Stream

Klassen: Reader, Writer → gut für Textdateien
Lesen:
try (var reader = new BufferedReader(
 new FileReader("text.txt", StandardCharsets.UTF_8))) {
 String line;
 while ((line = reader.readLine()) != null) {
 System.out.println(line); }
Schreiben: try (var writer = new FileWriter("text.txt",
 StandardCharsets.UTF_8)) {writer.write("Hallo Welt'n"); }
Objekt Serialisierung

Objekte speichern und wieder laden
// Serialisieren
try (var out = new ObjectOutputStream(
 new FileOutputStream("obj.bin"))) {
 out.writeObject(person); }

// Deserialisieren
try (var in = new ObjectInputStream(
 new FileInputStream("obj.bin"))) {
 Person p = (Person) in.readObject(); }

Lambdas

Ad-Hoc Methoden, anstelle von expliziten Methodenreferenzen
Syntax: (Parameterliste) -> (Body)
Sortieren nach alter:
people.sort((Person p1, Person p2) -> { return Integer.compare(p1.getAge(), p2.getAge()); }); //Ist Aufsteigend, für absteigend p1/p2 vertauschen rechts
Sortieren nach Nachname:
people.sort((p1, p2) -> p1.getLastName().compareTo(p2.getLastName()));
Sortieren Aufsteigend nach Ortschaft, Nachname und Vorname:
people.sort((p1, p2) -> {
 int value = p1.getCity().compareTo(p2.getCity());
 if (value == 0) {
 value = p1.getLastName().compareTo(p2.getLastName());
 }
 if (value == 0) {
 value = p1.getFirstName().compareTo(p2.getFirstName());
 }
 return value; }
Comparable

Wenn eigene Sortierkriterien gewünscht sind (z.B Collections.sort(people) reicht nicht). Objekt erkennt natürliche sortierung mit **compareTo()**

Integer Vergleich:
class Person implements Comparable {
 private int age;
 @Override
 public int compareTo(Person other) {
 return Integer.compare(age, other.age); } }

Namen vergleichen:
class Person implements Comparable {
 private final String firstName, lastName;
 @Override public int compareTo(Person other) {
 int result = lastName.compareTo(other.lastName);
 if (result == 0) { result = firstName.compareTo(other.firstName); }
 return result; } }

Zugriff von Lambda: pattern ist automatisch final
void test() { String pattern = readFromConsole();
 Utils.removeAll(people, p -> p.getLastName().contains(pattern)); ... }

Comparator

Externe Sortierregel wird beim Sortieren übergeben mit compare()

Anwendung: Collections.sort(people, new MyComparator());
class AgeComparator implements Comparator<Person> {
 @Override
 public int compare(Person p1, Person p2) {
 return Integer.compare(p1.getAge(), p2.getAge()); } }

Comparator Bausteine:

// Nach Alter
people.sort(Comparator.comparing(Person::getAge));
// Nach absteigendem Nachnamen
people.sort(
 Comparator.
 comparing(Person::getLastName)
 .reversed());
// Primär nach Nachnamen, sekundär nach Vornamen
people.sort(
 Comparator.
 .comparing(Person::getLastName)
 .thenComparing(Person::getFirstNameName);

Methodenreferenzen

Sort() mitgeben welche Methode sie zum Vergleich benutzen soll.

void print(List people) {
 people.sort(this::compareByAge); System.out.println(people); }

int compareByAge(Person p1, Person p2) {
 return Integer.compare(p1.getAge(), p2.getAge()); }

StreamAPI

Anwendung von Lambdas, wie SQL Abfragen, Anwendbar auf Collections. Bei Map muss zuerst values() auferufen werden.

people.stream()
 .filter(p -> p.getAge() >= 18) → WHERE
 .map(p -> p.getLastName()) → SELECT
 .sorted() → ORDER BY
 .forEach(System.out::println); → Terminaloperation

Stream zu Array: Person[] array = peopleStream.toArray(Person[]::new);
Stream zu Set: Set<Person> set = peopleStream.collect(Collectors.toSet());
Stream zu List: List<Person> list = peopleStream.toList();

Zwischenoperationen

filter(Predicate) → darf liste nicht abändern
- Elemente gemäss Bedingung auswählen
- entspricht WHERE
map(Function) → keine Abhängigkeit zu äusseren Variablen
- Elemente umwandeln / projizieren
- entspricht SELECT
mapToLong / mapToDouble
- Projektion auf primitive Typen (int, long, double)
flatMap(Function)
- Stream von Streams zu einem Stream machen
sorted(), sorted(Comparator)
- Sortieren mit natürlicher Ordnung oder Comparator
- entspricht ORDER BY
distinct()
- Duplikate entfernen gemäss equals()
limit(long n)
- Erste n Elemente behalten
skip(long n)
- Erste n Elemente überspringen

Terminaloperationen

forEach(Consumer): Pro Element Operation anwenden, meist mit Seiteneffekt
forEachOrdered(Consumer): Erhält die Reihenfolge der Elemente
count(): Anzahl Elemente
min(), max(): Mit Comparator Argument
average(), sum(): Nur bei Int/Long/Double Streams
findAny(), findFirst(): Gibt irgendein bzw. erstes Element zurück
allMatch(), anyMatch(), noneMatch(): Prüfen ob Prädikate auf alle/irgendein/kein Element zutrifft

Endliche Quellen

IntStream.range(1, 100): Zahlen von 1 bis 100 (analog für LongStream)
Stream.of(2, 3, 5, 7, 11, 13): Eigene Aufzählung
Stream.empty(): Leerer Stream (Testzwecke)
Stream.concat(stream1, stream2): Alle Elemente in stream 1, gefolgt von Elementen in stream2

```
Unendliche Quellen
Genererte:
Random random = new Random();
Stream.generate(random::nextInt).forEach(System.out::println);
Iterate: IntStream.iterate(0, i => i + 1, ()::forEach(System.out::println);
Auf Iterate kann man auch einen Filter anwenden z.B (.filter(number -> isPrime(number)))

Code Beispiele
Streams
// Namen aller Kaffees ausgeben
coffees.stream()
    .map(coffee -> coffee.getName())
    .forEach(System.out::println);

// b) Stärkste Kaffeemischung ermitteln
var strongest = coffees.stream()
    .max((c1, c2) -> Integer.compare(c1.getStrength(), c2.getStrength()));

// c) Namen aller Mischungen in aufsteigender Stärke ausgeben (Duplikate ignorieren)
coffees.stream()
    .sorted(Comparator.comparing(Coffee::getStrength))
    .map(coffee -> coffee.getName())
    .distinct()
    .forEach(System.out::println);

// d) Mischungen zählen, die Bohnen aus Vietnam enthalten
long fromVietnam = coffees.stream()
    .filter(coffee ->
        coffee.getBeans().stream()
            .anyMatch(bean -> bean.getOrigin().equals("Vietnam")))
    .count();

// b) Anzahl infizierter Humans zählen
long numberOfInfectedCollection<Human> humans {
    return humans.stream()
        .filter(Human::isInfected)
        .count(); // count() liefert long

// c) Gesamtzahl der Infektionen über alle Länder berechnen
int totalInfections(Map<String, Integer> infectionsByCountry) {
    return infectionsByCountry.values()
        .stream()
        .mapToInt(Integer::intValue)
        .sum();

// Alternative:
// .mapToInt(i -> i).sum();

// d) Humans mit einem Pathogen infizieren, falls sie nicht immun sind
public static void infectPeople(Collection<Human> humans, Pathogen pathogen) {
    humans.stream()
        .filter(human -> !human.isImmune(pathogen))
        .forEach(h -> {
            try {
                h.infect(pathogen);
            } catch (ImmuneException e) {
                // I kann hier nicht passieren!!!
            }
        });

// Durchschnittsalter pro Ort (Gruppierung mittels Collector)
people.stream()
    .collect(Collectors.groupingBy(Person::getCity, Collectors.averagingInt(Person::getAge))
    .forEach((city, age) -> System.out.println(city + " + " + age));

// Top 10 verdienner
people.stream()
    .sorted((p1, p2) -> Integer.compare(p2.getSalary(), p1.getSalary())) /h
    .limit(10)
    .map(p -> p.getLastName() + " + " + p.getFirstName())
    .forEach(System.out::println);

Klassen modellieren
-Hat die Klasse Felder oder gemeinsamen Zustand? -> abstract class
-Muss eine Klasse mehrere Rollen gleichzeitig haben? -> interface
-Ist es nur eine Fähigkeit/Regel ohne Zustand? -> interface
-Ist die Basisklasse vollständig nutzbar? -> normale Klasse
-Muss jede Unterklasse eigene Implementierung liefern? -> abstract Methode
-Wann muss eine Methode abstract sein? Jede Unterklasse muss die Methode haben, ist jedoch unklar wie umsetzen

/* Tourist Attractions – Vererbung + Polymorphie */
public abstract class TouristAttraction {
    public abstract void printAction();
}

public class Restaurant extends TouristAttraction {
    private int numberOfTables;
    @Override
    public void printAction() {
        System.out.println("eat in a restaurant");
    }
}

public class Pizzeria extends Restaurant {
    @Override
    public void printAction() {
        System.out.println("eat a pizza");
    }
}

public class Shop extends TouristAttraction {
    @Override
    public void printAction() {
        System.out.println("buy something in the store");
    }
}

public class ShoppingMail extends TouristAttraction {
    private ArrayList<TouristAttraction> attractions;
    @Override
    public void printAction() {
        for (var attraction : attractions) {
            attraction.printAction();
        }
    }
}
```

```
Quersumme
/* Quersumme berechnen */
int digitSum(int number) {
    int result = 0;
    while (number > 0) {
        result += number % 10; // letzte Ziffer addieren
        number /= 10; // letzte Ziffer entfernen
        return result;
    }
}

/* Einstellige Quersumme (digitale Wurzel ohne Formel) */
int singleDigitSum(int number) {
    while (number > 9) { // solange mehrstellig
        number = digitSum(number);
        return number;
    }
}

/* Prüft, ob die beiden Diagonalen die gleiche Summe haben */
boolean isMagicSquare(int[][] values) {
    int sum1 = 0;
    int sum2 = 0;
    for (int i = 0; i < values.length; i++) {
        sum1 += values[i][i]; // links oben -> rechts unten
        sum2 += values[i][values.length - 1 - i]; // links unten -> rechts oben
    }
    return sum1 == sum2;
}

Sortierung
/*Aufgabe: Ein Array ist "fast sortiert".
Genau ein Element ist an der falschen Position.
Beispiel: [2, 3, 5, 8, 5, 7, 9] -> 8 ist falsch

Ziel:
a) Finde den Index des falsch platzierten Elements.
b) Sortiere das Array, indem dieses Element an die richtige Stelle verschoben wird.
Ohne Sortier- oder Suchmethoden aus der API!

/* a) Index des falsch platzierten Elements finden */
int unsortedIndex(int[] array) {
    for (int i = 0; i < array.length - 1; i++) {
        if (array[i] > array[i + 1]) {
            if (i > 0 && array[i - 1] <= array[i + 1]) {
                return i;
            } else if (i < array.length - 2 && array[i] <= array[i + 2]) {
                return i + 1;
            } else if (i > 0) {
                return i - 1;
            } else {
                return i;
            }
        }
    }
    throw new AssertionError("All sorted");
}

/* b) Korrekt einsortieren mit Entfernen + Einfügen */
void delete(int[] array, int position) {
    for (int i = position; i < array.length - 1; i++) {
        array[i] = array[i + 1];
    }
    array[array.length - 1] = Integer.MAX_VALUE;
}

void insert(int[] array, int value) {
    int position = 0;
    while (position < array.length && value > array[position]) {
        position++;
    }
    for (int i = position; i < array.length - 1; i++) {
        array[i + 1] = array[i];
    }
    array[position] = value;
}

void resort(int[] array) {
    int index = unsortedIndex(array);
    int value = array[index];
    delete(array, index);
    insert(array, value);
}

Rekursion
/* Zählt alle Mitarbeiter inkl. Unterabteilungen (rekursiv) */
int countEmployees() {
    int count = employees.size();
    for (var sub : subunits()) {
        count += sub.countEmployees();
    }
    return count;
}

/*Summiert alle Kalorien verschachtelt*/
class ProcessedFood extends Food {
    private final List<Food> ingredients = new ArrayList<>();
    public ProcessedFood(String name) {
        super(name);
    }
    @Override
    public int getCalories() {
        int calories = 0;
        for (Food food : ingredients) {
            calories += food.getCalories();
        }
        return calories;
    }
}

Erinnerungsfotos (kinda wie linked list)
/*Erinnerungsfotos:
-Keine Arrays / Collections erlaubt
-Eigene Datenstruktur (einfach verkettete Liste)
-Sortiert nach Datum: ältestes Bild zuerst*/
public class ImageViewer {
    private Image head;
    private int size = 0;
    // fügt Bild sortiert nach Datum ein
    public void add(Date dateRecorded, byte[] data) {
        Image newImage = new Image(dateRecorded, data);
        Image current = head;
        Image previous = null;
        while (current != null && dateRecorded.after(current.dateRecorded)) {
            previous = current;
            current = current.next;
        }
        if (previous == null) {
            head = newImage;
        } else {
            previous.next = newImage;
            newImage.next = current;
            size++;
        }
        // entfernt Bild an Index und gibt es zurück
        public Image remove(int index) {
            if (index < 0 || index >= size) {
                throw new IndexOutOfBoundsException();
            }
            Image current = head;
            Image previous = null;
            int pos = 0;
            while (pos < index) {
                previous = current;
                current = current.next;
                pos++;
            }
            if (previous == null) {
                head = current.next;
            } else {
                previous.next = current.next;
            }
            return current;
        }
        public int getSize() {
            return size;
        }
        class Image {
            Date dateRecorded;
            byte[] data;
            Image next;
            public Image(Date dateRecorded, byte[] data) {
                this.dateRecorded = dateRecorded;
                this.data = data;
            }
        }
    }
}

RingBuffer
public class RingBuffer<T> {
    private Object[] elements;
    private int writePosition = 0;
    private int readPosition = 0;
    private int size = 0;
    public RingBuffer(int capacity) {
        elements = new Object[capacity];
    }
    public RingBuffer(int capacity, T... initial) {
        elements = new Object[capacity];
        for (T x : initial) write(x);
    }
    public void write(T element) {
        elements[writePosition] = element;
        writePosition++;
        size++;
        if (writePosition == elements.length) {
            writePosition = 0;
        }
        if (size > elements.length) {
            size = elements.length;
            readPosition++;
            if (readPosition == elements.length) {
                readPosition = 0;
            }
        }
    }
    public T read() {
        if (size == 0) return null;
        T element = (T) elements[readPosition];
        readPosition++;
        size--;
        if (readPosition == elements.length) {
            readPosition = 0;
        }
    }
    public int size() {
        return size;
    }
    public boolean isEmpty() {
        return size == 0;
    }
    public boolean isFull() {
        return size == elements.length;
    }
}

Neue Elemente aus Rückgabe einer Methode erzeugen
/* Erzeugt Podcast aus Byte-Arrays der Kapitel */
public Podcast createPodcast(ArrayList<byte[]> chapters) {
    var chapterObjects = new ArrayList<Chapter>();
    for (var chapter : chapters) {
        chapterObjects.add(new Chapter(chapter));
    }
    return new Podcast(chapterObjects);
}

Aufnahme spliten anhand von Kapitelmarker (Podcast)
/* Teilt Aufnahme anhand der Kapitelmarken in einzelne Kapitel */
ArrayList<byte[]> splitIntoChapters(byte[] recording, int[] chapterMarks) {
    var result = new ArrayList<byte[]>();
    int previousChapterEnd = 0;
    for (int nextChapterEnd : chapterMarks) {
        byte[] nextChapter = Arrays.copyOfRange(recording, previousChapterEnd, nextChapterEnd);
        nextChapterEnd = nextChapterEnd;
        result.add(nextChapter);
        previousChapterEnd = nextChapterEnd;
    }
    byte[] lastChapter = Arrays.copyOfRange(recording, previousChapterEnd, recording.length);
    result.add(lastChapter);
    return result;
}

Merge zwei Filme
Movie[] mergeWatchLists(Movie[] listA, Movie[] listB) {
    public void add(Date dateRecorded, byte[] data) {
        Movie newImage = new Movie(listA.length + listB.length);
        int i = 0, a = 0, b = 0;
        // merge while both arrays contain elements
        while (a < listA.length && b < listB.length) {
            if (listA[a].getDuration() <= listB[b].getDuration()) {
                merged[i++] = listA[a++];
            } else {
                merged[i++] = listB[b++];
            }
            // copy remaining elements from A
            while (a < listA.length) {
                merged[i++] = listA[a++];
            }
            // copy remaining elements from B
            while (b < listB.length) {
                merged[i++] = listB[b++];
            }
            return merged;
        }
    }
}

Print mit indent
Spaghetti Napoli (124 Kalorien)
Spaghetti (100 Kalorien)
Tomatensauce (24 Kalorien)
Tomate (10 Kalorien)
Paprika (12 Kalorien)
Salz (0 Kalorien)
Pfeffer (2 Kalorien)
Wasser (0 Kalorien)

abstract class Food {
    public abstract String toString(indent);
    @Override
    public String toString() {
        return toString(0);
    }
    protected String getIndent(int length) {
        String text = "";
        for (int i = 0; i < length; i++) text += " ";
        return text;
    }
}

class BasicFood extends Food {
    @Override
    public String toString(indent) {
        return getIndent(indent) + getName() + " (" + getCalories() + " Kalorien)";
    }
}

class ProcessedFood extends Food {
    @Override
    public String toString(indent) {
        String text = getIndent(indent) + getName() + " (" + getCalories() + " Kalorien)";
        for (Food f : ingredients) {
            text += f.toString(indent + 1);
        }
        return text;
    }
}

Diff-Tool für Files (Input/Output)
/* Diff-Tool: vergleicht zwei Personendateien */
void compare(String filePath1, String filePath2) throws IOException {
    Map<Integer, String> entries1 = readFile(filePath1);
    Map<Integer, String> entries2 = readFile(filePath2);

    // Einträge aus Datei 1 prüfen
    for (int number : entries1.keySet()) {
        String name1 = entries1.get(number);
        String name2 = entries2.get(number);
        if (name2 == null) {
            System.out.println("Fehlt in zweiter Datei: " + name + " + " + name1);
        } else if (!name1.equals(name2)) {
            System.out.println("Unterschied: " + name + " + " + name1 + " <" + name + " + " + name2);
        }
    }
    // Einträge, die nur in Datei 2 existieren
    for (int number : entries2.keySet()) {
        if (!entries1.containsKey(number)) {
            System.out.println("Fehlt in erster Datei: " + name + " + " + entries2.get(number));
        }
    }
}

Map<Integer, String> readFile(String filePath) throws IOException {
    Map<Integer, String> entries = new HashMap<>();
    try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
        String line;
        while ((line = reader.readLine()) != null) {
            int number = Integer.parseInt(line.substring(0, 3));
            String name = line.substring(4);
            entries.put(number, name);
        }
    }
    return entries;
}

Max Clique
/*Finde die grösste Personengruppe, in der jede Person mit jeder anderen befreundet ist.*/
Set<Person> maxClique() {
    Set<Person> result = new HashSet<>();
    for (Set<Person> group : allSubsets(new ArrayList<>(allPeople))) {
        if (!isClique(group) && result.size() < group.size()) {
            result = group;
        }
    }
    return result;
}

boolean isClique(Set<Person> group) {
    for (Person first : group) {
        for (Person second : group) {
            if (first != second && !first.getFriends().contains(second)) {
                return false;
            }
        }
    }
    return true;
}

Set<Set<Person>> allSubsets(List<Person> people) {
    Set<Set<Person>> result = new HashSet<>();
    result.add(new HashSet<>());
    for (Person current : people) {
        List<Person> remaining = new ArrayList<>(people);
        remaining.remove(current);
        for (Set<Person> subset : allSubsets(remaining)) {
            Set<Person> augmented = new HashSet<>(subset);
            augmented.add(current);
            result.add(augmented);
        }
    }
    return result;
}

Zerfallsreihe
class Isotope {
    private final String element;
    private final int neutrons;
    private final Set<Isotope> decayProducts = new HashSet<>();
}

// Konstruktor, Getter, Setter
// b) direkte und indirekte Zerfallsprodukte
Set<Isotope> transitiveDecayProducts(Isotope current) {
    Set<Isotope> result = new HashSet<>();
    for (Isotope directProduct : current.getDecayProducts()) {
        result.add(directProduct);
        for (Isotope indirectProduct : transitiveDecayProducts(directProduct)) {
            result.add(indirectProduct);
        }
    }
    return result;
}

// c) Zyklenerkennung
void checkAcyclic(Isotope current) {
    checkAcyclic(current, new HashSet<>());
    void checkAcyclic(Isotope current, Set<Isotope> visited) {
        if (visited.contains(current)) {
            throw new RuntimeException("Cyclic!");
        }
        visited.add(current);
        for (Isotope follower : current.getDecayProducts()) {
            checkAcyclic(follower, new HashSet<>(visited)); // Kopie wegen DAG-Struktur!
        }
    }
}

Theoriemix
/* Wahr machen der Ausdrücke */
a + a * 2 == 12 -> a = 4
b / 2 == b -> b = 0
!c -> c = false
d.length == d[0] -> d = new int[] { 1 }
e != 0 && e / 2 == (e + 1) / 2 -> e = jede ungerade negative int (oder Float.POSITIVE_INFINITY)
f + "" != f -> f = "abc" // jeder String
g + 1 == g + 2 -> g = Float.POSITIVE_INFINITY
h != h -> h = Float.NaN

Stolpersteine:
- Overloading: Methoden mit dem gleichen Namen (Identifier), aber unterschiedlicher Parameterliste (unterschiedliche Anzahl oder Typen der Parameter). Statische Auflösung durch den Compiler
- Overriding: Methode mit gleicher Signatur (gleicher Identifier und Parameterliste) wird in der Subklasse oder der implementierten Schnittstelle erneut implementiert. Dynamische Auflösung durch virtual Call/Dynamic Dispatch durch Laufzeitsystem.
- Overloading oder Overriding? Compiler sieht zwei passende Methoden im statischen Typ und wählt die spezifischere. Compiler sieht passende Methode im dynamischen Typ -> Methode wird überschrieben,
-Wieso soll man Strings nicht mit == vergleichen? Verschiedene String-Objekte können den gleichen Text haben (z.B. bei Einlesen von I/O u.a. werden jeweils neue String Objekte angelegt). So sind die Referenzen unterschiedlich (Referenzvergleich == ergibt false), obwohl die Texte der Strings dennoch gleich. Vergleich mit equals() wird daher empfohlen.
-Wieso soll man float und double nicht mit == vergleichen? Numerische Fehler wegen der Maschinenpräzision sind möglich und akkumulieren sich bei Operationen. So sind analytisch gleiche Formeln als numerische Floating Point Ausdrücke allenfalls ungleich (z.B. ist 0.1 + 0.1 + 0.1 != 0.3). Ein Vergleich mit einer Fehlertoleranz (Epsilon) (Math.abs(x - y) < epsilon) wird empfohlen.
Lambda-Beispiel:
(String a, String b, String c) -> {
    if (a.length() < b.length()) {
        if (c.length() <= b.length()) {
            return false;
        } else {
            return true;
        }
    } else {
        return false;
    }
}

wird zu: (a, b, c) -> a.length() < b.length() && b.length() < c.length()
Generische Funktionsschnittstelle für das vorherige Lambda:
interface TriplePredicate<T> {
    boolean test(T first, T second, T third);
}

Aufruf vom vorherigen Lambda:
TriplePredicate<String> predicate =
    (a, b, c) -> a.length() < b.length() && b.length() < c.length();
boolean result = predicate.test("Paul", "Peter", "Clemens");
```

<pre>class Train { void append(Train tail) { // Method 1 } void append(FastTrain tail) { // Method 2 } }</pre>	<pre>class FastTrain extends Train { void append(Train tail) { // Method 3 } void append(FastTrain tail) { // Method 4 } }</pre>														
Bestimmen Sie, welche Methode jeweils ausgeführt wird:															
Train a = new FastTrain(); FastTrain b = new FastTrain();															
<table><tr><th>Aufruf</th><th>Methode</th></tr><tr><td>a.append(null);</td><td>Method 4</td></tr><tr><td>a.append(b);</td><td>Method 4</td></tr><tr><td>a.append(Train bb);</td><td>Method 3</td></tr><tr><td>b.append(null);</td><td>Method 4</td></tr><tr><td>b.append(a);</td><td>Method 4</td></tr><tr><td>((Train bb).append(a));</td><td>Method 3</td></tr></table>	Aufruf	Methode	a.append(null);	Method 4	a.append(b);	Method 4	a.append(Train bb);	Method 3	b.append(null);	Method 4	b.append(a);	Method 4	((Train bb).append(a));	Method 3	
Aufruf	Methode														
a.append(null);	Method 4														
a.append(b);	Method 4														
a.append(Train bb);	Method 3														
b.append(null);	Method 4														
b.append(a);	Method 4														
((Train bb).append(a));	Method 3														